
Internet das Coisas - 2025/2026

PROJETO FINAL

SatMon - Monitorização de uma Constelação de Satélites

Daniela Correia - 2024153821

Tiago Cardoso - 2024153832

Mestrado em Engenharia Eletrotécnica e de Computadores
Departamento de Engenharia Eletrotécnica e de Computadores
Faculdade de Ciências e Tecnologia da Universidade de Coimbra

Índice

1. Introdução.....	3
2. Arquitetura do Sistema e Tecnologias	3
3. Aquisição e Processamento nos Nós IoT.....	4
4. Comunicações, Protocolos e Integração Cloud.....	4
5. Interface com o Utilizador e Telecomando	5
6. Resultados e Validação Experimental	6
7. Escalabilidade, Limitações e Potencial	6
8. Desvios face ao Planeado	7
9. Conclusão	7
Referências.....	8

1. Introdução

As constelações de satélites são sistemas distribuídos nos quais vários veículos geram continuamente telemetria sobre o seu estado operacional. A monitorização simultânea destes nós exige mecanismos de aquisição, comunicação, armazenamento, análise e telecomando. Neste contexto, o SatMon implementa um protótipo IoT que simula uma constelação de pequenos satélites através de placas ESP32-C6, integradas com uma infraestrutura local e serviços cloud da AWS.

Cada nó representa um satélite e adquire dados de orientação e temperatura. A telemetria é publicada periodicamente por MQTT, encaminhada para a cloud, armazenada e disponibilizada ao operador através de um dashboard de missão. O sistema permite ainda o envio de comandos no sentido inverso, nomeadamente o controlo remoto de um LED RGB instalado em cada placa, reproduzindo de forma simplificada o ciclo de telemetria e telecomando de um sistema.

O objetivo principal é demonstrar uma solução completa e escalável, desde a aquisição dos dados dos sensores até à apresentação de informação útil. Para além dos valores em bruto, o SatMon estima a atitude do nó, deteta condições de temperatura e comunicação anómalas, apresenta histórico e permite atuar remotamente sobre cada satélite.

2. Arquitetura do Sistema e Tecnologias

A arquitetura final está organizada em três camadas, os nós IoT, a infraestrutura local executada em Docker containers e os serviços cloud AWS. A telemetria percorre o sistema dos nós para a cloud; os comandos percorrem o caminho inverso. A Figura 1 apresenta os principais componentes e os fluxos de comunicação.

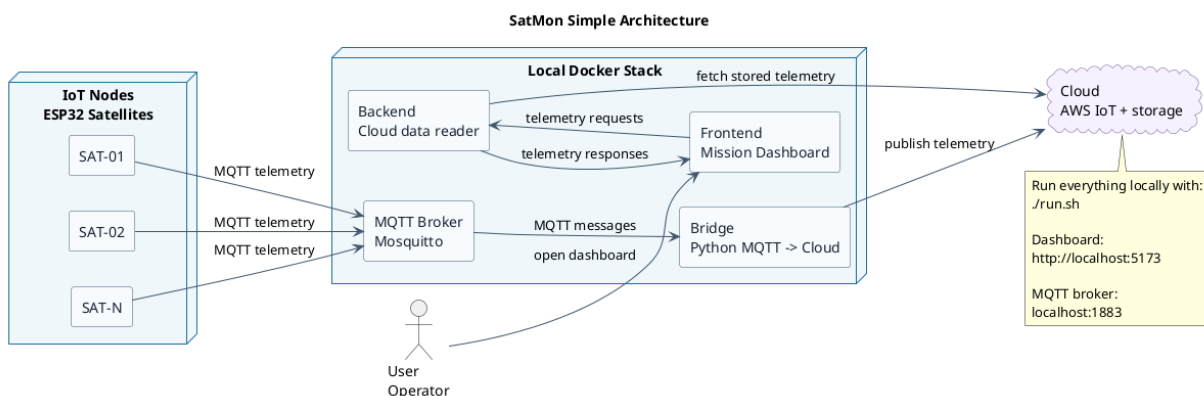


Figura 1 - Arquitetura final do sistema SatMon.

Módulo	Tecnologia	Linguagem / ambiente	Função
Nós IoT	ESP32-C6, IMU, WS2812	C, ESP-IDF, FreeRTOS	Aquisição, telemetria e atuação local

Broker local	Eclipse Mosquitto	Docker	Distribuição MQTT na rede local
Ponte cloud	Paho MQTT	Python, Docker	Encaminhamento bidirecional local-cloud
Ingestão cloud	AWS IoT Core	AWS	Receção MQTT e regras de encaminhamento
Processamento	AWS Lambda	Python / boto3	Validação, persistência e comandos
Armazenamento	Amazon DynamoDB	AWS NoSQL	Histórico indexado por satélite e tempo
API	API Gateway + Lambda	HTTPS REST / JSON	Consulta, manutenção e telecomando
Dashboard	HTML, CSS, JavaScript, Three.js	Browser + servidor Python	Visualização e interação do operador

3. Aquisição e Processamento nos Nós IoT

Cada satélite é representado por uma placa ESP32-C6, microcontrolador RISC-V de 32 bits com Wi-Fi. A unidade de medição inercial é ligada por I2C, utilizando SDA no GPIO 6, SCL no GPIO 7 e frequência de 400 kHz. O firmware identifica automaticamente o sensor através do registo WHO_AM_I e suporta módulos das famílias MPU-92/65 e LSM6DS, permitindo substituir a IMU sem alterar a arquitetura do software.

Em cada ciclo são adquiridas a aceleração nos três eixos, a velocidade angular nos três eixos e a temperatura interna da IMU. No arranque é efetuada uma calibração do desvio (bias) de zero do giroscópio com 64 amostras. A taxa de publicação por omissão é de uma amostra por segundo. O timestamp UTC é obtido por SNTP e cada mensagem inclui o identificador “satellite_id”.

A atitude é estimada através de um filtro complementar que combina a integração das velocidades angulares do giroscópio com os ângulos de inclinação obtidos pelo acelerómetro. Esta fusão estabiliza as estimativas de roll e pitch. Na ausência de magnetómetro, o yaw é obtido principalmente por integração do giroscópio e está sujeito a deriva acumulada, uma limitação considerada aceitável para o protótipo.

O LED RGB integrado no próprio MCU, serve para demonstrar o envio de telecomandos. O firmware foi desenvolvido em C com ESP-IDF/FreeRTOS; uma tarefa dedicada lê a IMU, constrói o JSON e publica-o através do cliente esp-mqtt. Perfis de configuração definem SAT-01, SAT-02 ou um identificador personalizado. O software pode também ser executado em QEMU para testes sem hardware físico.

4. Comunicações, Protocolos e Integração Cloud

Na rede local, os nós ligam-se por Wi-Fi/IPv4 ao broker Mosquitto e publicam telemetria por MQTT sobre TCP com QoS 1. O formato de mensagem é JSON. A utilização de QoS 1 garante entrega pelo menos uma vez, sendo possíveis duplicados, que são tolerados pelo processamento baseado em timestamp. A ligação local pode ser protegida por TLS, recorrendo a bundle de certificados, CA embebida ou certificado próprio.

A ponte cloud é um serviço Python baseado em paho-mqtt. O serviço subscreve os tópicos locais, preserva o id do satélite e encaminha a telemetria para o destino configurado. Estão disponíveis três modos, o registo local para desenvolvimento, HTTP POST e AWS IoT Core. No modo AWS, a ligação utiliza MQTT sobre TLS 1.2, autenticação mútua por certificados X.509 e publicação no espaço de tópicos SatMon.

Na região eu-west-1, uma regra do AWS IoT Core invoca a função Lambda de processamento sempre que chega uma mensagem. A função valida o payload e grava a amostra na tabela DynamoDB, cuja chave de partição é satellite_id e cuja chave de ordenação é timestamp. Uma segunda Lambda, exposta através de API Gateway, implementa as operações de leitura, histórico, eliminação de telemetria e envio de comandos.

Ligação	Protocolo	Segurança	Formato / cadência
ESP32-C6 -> Mosquitto	MQTT/TCP, QoS 1	Wi-Fi; TLS opcional	JSON, 1 mensagem/s/nó
Ponte -> AWS IoT Core	MQTT/TLS 1.2	X.509 mútuo	JSON, orientado a eventos
IoT Core -> Lambda	AWS IoT Rule	IAM / permissões AWS	Invocação por mensagem
Lambda -> DynamoDB	AWS SDK / boto3	IAM least privilege	Uma escrita por amostra
Dashboard -> API	HTTPS REST	TLS, CORS/proxy local	JSON, polling a cada 1 s
Cloud -> ESP32-C6	MQTT de comando	TLS no segmento cloud	JSON, sob pedido

A atualização do dashboard é realizada por polling periódico da API REST, com intervalo de um segundo. Assim, a ingestão da telemetria é orientada a eventos através de MQTT, enquanto a interface apresenta informação em tempo quase real, com atraso máximo aproximado correspondente ao período de polling, acrescido da latência de rede e processamento.

5. Interface com o Utilizador e Telecomando

O dashboard é uma aplicação web desenvolvida em HTML, CSS e JavaScript, utilizando Three.js para a representação tridimensional da atitude. É servido por um pequeno servidor Python que atua também como proxy da API REST, centralizando o endereço do serviço e evitando dependência direta das políticas CORS no browser.

A interface apresenta uma vista global da constelação, cartões por satélite, indicadores agregados, tabela de telemetria, histórico de temperatura, estado de ligação e modelo 3D. O operador consegue identificar rapidamente nós ativos, atrasados ou desligados, bem como situações de aviso ou anomalia térmica.

O telecomando permite selecionar uma cor e ativar ou desativar o LED de um satélite específico ou de todos os nós. O dashboard envia um pedido POST para a API; a Lambda publica o comando no AWS IoT Core; a ponte recebe-o e republica-o no broker local; e o firmware aplica o comando. Esta cadeia demonstra comunicação bidirecional e fecha o ciclo de monitorização e controlo.

6. Resultados e Validação Experimental

A validação foi realizada de forma incremental, começando pelos sensores e comunicação local e terminando no fluxo completo cloud-dashboard. Foram testadas mensagens de funcionamento normal, condições térmicas de aviso e anomalia, perda de comunicação e comandos LED individuais e em difusão.

Ensaio	Resultado esperado	Resultado observado	Estado
Aquisição da IMU	Aceleração, giroscópio e temperatura válidos	Leituras periódicas e identificação automática da IMU	Aprovado
Publicação MQTT local	Mensagem JSON recebida pelo Mosquitto	Telemetria recebida a 1 Hz com QoS 1	Aprovado
Encaminhamento para AWS	Mensagem entregue ao AWS IoT Core	Regra IoT invocou a Lambda de processamento	Aprovado
Persistência	Amostra indexada por satélite e timestamp	Registos consultáveis na DynamoDB	Aprovado
API REST	Última amostra e histórico devolvidos em JSON	Endpoints consumidos pelo dashboard	Aprovado
Aviso/anomalia térmica	Alerta aos 45 °C / 55 °C	Classificação apresentada na interface	Aprovado
Nó offline	Estado alterado após atraso de telemetria	Dashboard sinalizou ligação atrasada/desligada	Aprovado
Telecomando LED	Comando aplicado no nó selecionado	Cor/estado alterados individualmente e em difusão	Aprovado

Os testes confirmaram o funcionamento de ponta a ponta: aquisição no nó, publicação no broker local, encaminhamento seguro para a AWS, armazenamento, consulta pela API e visualização no dashboard. O canal inverso foi igualmente validado através do controlo do LED. A cadência de 1 Hz e o polling de 1 s são adequados ao objetivo de demonstração e fornecem uma experiência de monitorização quase em tempo real.

7. Escalabilidade, Limitações e Potencial

Escalabilidade. Os nós estão desacoplados por MQTT e identificam-se autonomamente. A ponte aceita novos tópicos sem configuração central e a adição de um satélite resume-se a gravar o firmware com um novo `satellite_id`. Na cloud, a DynamoDB distribui os dados pela chave de partição e as funções Lambda escalam automaticamente com a carga. Os comandos podem ser dirigidos a um nó ou publicados em difusão.

Utilidade: O sistema transforma medições em informação operacional: estima orientação, mostra tendências, calcula indicadores agregados e gera alertas térmicos e de ligação. O operador recebe uma visão acionável da constelação, em vez de apenas valores sensoriais isolados.

Limitações: O yaw acumula deriva por ausência de uma referência absoluta de rumo; o dashboard usa polling e não comunicação push; e a validação foi realizada à escala de protótipo. A disponibilidade depende ainda da conectividade Internet e das credenciais AWS. Estas limitações não comprometem os objetivos didáticos, mas devem ser consideradas numa evolução para operação real.

Potencial: A arquitetura pode ser aplicada a frotas de sensores, veículos, monitorização ambiental ou telemetria industrial. Evoluções possíveis incluem autenticação de utilizadores, WebSocket/AppSync para atualização push, deteção de anomalias por aprendizagem automática, atualizações OTA, magnetómetro para correção de yaw, encriptação TLS também no broker local e políticas IAM mais restritivas.

8. Desvios face ao Planeado

Alteração	Motivação	Impacto
Camada local Docker com broker e ponte	Permitir desenvolvimento e testes independentes da cloud	Maior modularidade e reprodutibilidade
Ponte com destinos log/HTTP/AWS	Facilitar depuração e evolução progressiva	Redução do risco durante a integração
Deteção automática de várias famílias de IMU	Aumentar compatibilidade de hardware	Firmware mais reutilizável
Telecomando bidirecional do LED	Demonstrar atuação remota e não apenas monitorização	Maior valor funcional do protótipo
Dashboard exclusivamente via API REST	Separar interface, armazenamento e MQTT	Arquitetura mais clara e segura

As alterações mantiveram o conceito inicial e reforçaram a capacidade de teste, compatibilidade, modularidade e demonstração. O aumento de complexidade foi compensado por uma arquitetura mais próxima de uma solução IoT real.

9. Conclusão

O SatMon demonstra de ponta a ponta um sistema IoT para monitorização e telecomando de uma constelação simulada: aquisição inercial e térmica em ESP32-C6, transporte MQTT, integração segura com AWS IoT Core, processamento serverless, armazenamento em DynamoDB e apresentação numa interface de missão. O protótipo foi validado nos fluxos de telemetria, persistência, consulta, alertas, deteção de perda de ligação e comando remoto do LED. O vídeo demonstrativo está disponível em [11].

Os objetivos foram cumpridos com uma solução modular, extensível e escalável. A separação entre nós, infraestrutura local, cloud e dashboard permitiu testar cada componente isoladamente e integrar o sistema de forma progressiva. Embora existam limitações, nomeadamente a deriva de yaw e a atualização por polling, a implementação constitui uma base sólida para aplicações reais de monitorização de sistemas distribuídos.

Referências

- [1] Espressif Systems, ESP-IDF Programming Guide - ESP32-C6. Disponível em: <https://docs.espressif.com/projects/esp-idf/> (consultado em junho de 2026).
- [2] OASIS, MQTT Version 3.1.1 / 5.0 Specification. Disponível em: <https://mqtt.org> (consultado em junho de 2026).
- [3] Eclipse Foundation, Eclipse Mosquitto MQTT Broker. Disponível em: <https://mosquitto.org> (consultado em junho de 2026).
- [4] Eclipse Foundation, Eclipse Paho Python Client. Disponível em: <https://www.eclipse.org/paho/> (consultado em junho de 2026).
- [5] Amazon Web Services, AWS IoT Core Developer Guide. Disponível em: <https://docs.aws.amazon.com/iot/> (consultado em junho de 2026).
- [6] Amazon Web Services, AWS Lambda Developer Guide. Disponível em: <https://docs.aws.amazon.com/lambda/> (consultado em junho de 2026).
- [7] Amazon Web Services, Amazon DynamoDB Developer Guide. Disponível em: <https://docs.aws.amazon.com/amazondynamodb/> (consultado em junho de 2026).
- [8] Amazon Web Services, Amazon API Gateway Developer Guide. Disponível em: <https://docs.aws.amazon.com/apigateway/> (consultado em junho de 2026).
- [9] Three.js, JavaScript 3D Library. Disponível em: <https://threejs.org> (consultado em junho de 2026).
- [10] Docker Inc., Docker Compose Documentation. Disponível em: <https://docs.docker.com/compose/> (consultado em junho de 2026).
- [11] Vídeo do projeto, <https://youtu.be/wnGUt72cLm8>